

HelixLLM

HelixLLM

Један бинарни фајл, шест режима — OpenAI- и Anthropic-компатибилно закључивање од лаптопа до кластера са више хостова.

Сажетак

HelixLLM је дистрибуирани LLM систем за предузећа направљен у Go: један бинарни фајл са системом режима који се скалује од развоја на једном хосту до продукције на више хостова. Пружа потпуно OpenAI- и Anthropic-компатибилне API-је преко HTTP/3, са локалним закључивањем путем лама.цпп, ланцом резервних решења са оцењивањем више провајдера, RAG протоком и системом ReAct агената.

Кратак опис

HelixLLM је дистрибуирани LLM систем заснован на једном бинарном фајлу и Go. Излаже OpenAI- и Anthropic-компатибилне API-је преко HTTP/3, покреће локално закључивање путем лама.цпп, аутоматски открива и оцењује бесплатне провајдере у облаку како би формирао ланац резервних решења, те додаје RAG проточни систем знања и ReAct агента са позивањем алата — расположив у шест режима имплементације.

Детаљан опис

HelixLLM је дистрибуирани LLM систем за предузећа изграђен у Go са Gin, а његова главна предност је да један артефакт покрива све сценарије. Компајлира се у један бинарни фајл чији систем режима одређује приликом имплементације шта тај фајл *заправо јесте*: покрените га као `full` за свеобухватну инстанцу на лаптопу или поделите одговорности између

режима gateway, brain, knowledge, agents и control распоређених на више хостова — исти код, само преуређен, а не преправљен, од развојног рачунара до продукцијског кластера.

Говори два дијалекта течно: потпуно OpenAI- и Anthropic-компатибилне API-је, тако да постојећи SDK клијенти из оба екосистема раде непромењено, а све се сервира преко HTTP/3 (QUIC) са аутоматским HTTP/2 резервним решењем и TLS 1.3. Локално закључивање покреће се преко лама.цпп са подршком за CUDA, Метал и ROCm, па исти буилд убрзава рад на Nvidia, Apple и AMD хардверу. Најистакнутија карактеристика је ланац резервних решења са више провајдера, који претвара озлоглашену непоузданост бесплатних облака за закључивање у управљиви, самолечећи ресурс: HelixLLM аутоматски открива бесплатне моделе од преко 7 провајдера у облаку (Chutes, OpenRouter, ХуггингФаци, Nvidia, Cerebras, SambaNova, Тогеџер), оцењује их путем LLMsVerifier на сваких пет минута и усмерава саобраћај кроз ранжирани ланац са аутоматским резервним решењем за грешке 429/5xx — увек са локалним лама.цпп као гарантованом последњом опцијом, тако да захтев никада не пропадне само због недоступности провајдера.

Поред самог закључивања, HelixLLM је потпуна апликативна платформа: RAG проточни систем знања (унос, сегментација, уградња, вектор претрага) и систем ReAct агената са позивањем алата, сесијама разговора и интеграцијом RAG испоручују се у истом бинарном фајлу. Систем режима се исплати и на мрежном нивоу — у full режиму сви слојеви комуницирају директним Go позивима унутар процеса без мрежног оптерећења, док исти бинарни фајл, распоређен на више хостова, координира преко gRPC, SSE и Kafka. Заокружују га преговарање о садржају са Brotli/гзип компресијом, SSE стримовање које бајт-по-бајт одговара форматима OpenAI и Anthropic, аутентификација путем API кључа и JWT са ограничењем броја захтева, Prometheus метрике, OpenTelemetry праћење и велики скуп Go подмодула за продукцијску инфраструктуру.

Зашто смо га направили

Тимовима је потребан закључак који је преносив, компатибилан са стандардима и отпоран – без преписивања клијената или зависности од једног провајдера или једне машине. HelixLLM је направљен тако да исти бинарни фајл може да ради локално током развоја и да се скалује на кластер у продукцији са више хостова, говорећи дијалектима OpenAI и Anthropic које клијенти већ користе.

Зашто је револуционаран

Он сажима читав стек закључивања – гејтвеј, локално закључивање, резервни облак, RAG и агенте – у један бинарни фајл контролисан прекидачем режима, тако да архитектура коју имплементирате постаје одлука у време извршавања, а не пројекат поновне платформизације. Такође претвара оно што је раније било слабост у предност: поузданост провајдера у облаку постаје примарна, непрестано мерена брига, коју решава оцењивани, самолечећи ланац резервних решења који сваких неколико минута поново рангира провајдере и увек деградира на загарантовано локално закључивање. Оно што то омогућава јесте једна крајња тачка на коју се можете заиста ослонити – компатибилна са стандардима, преносива са лаптопа на кластер и неспособна да се угаси јер је неки узводни провајдер ограничио број захтева или отказао.

Шта је иновативно

- Један бинарни фајл са системом од шест режима који ради све-у-једном или као дистрибуиране улоге – директни позиви Go у процесу у режиму full, gRPC/SSE/Kafka када је подељен – тако да се топологија имплементације мења без измене кода или непотребног оптерећења мреже.
- Оцењивани, аутоматски откривајући ланац резервних решења са више провајдера преко 7+ бесплатних провајдера, непрестано ранжираних према LLMsVerifier са аутоматским пребацивањем на резерву при грешкама 429/5xx и загарантованим ллама.цпп као последњим решењем – бесплатни капацитет претворен у поуздан капацитет.
- Двоструке површине компатибилне са OpenAI и Anthropic, сервисирани преко HTTP/3 са аутоматским пребацивањем на HTTP/2, тако да се клијенти из оба екосистема повезују без модификација.
- Локално закључивање које покрива CUDA, Метал и ROCm из једног кодног извора – исти буилд ради убрзано на Nvidia, Apple и AMD хардверу.

Највећи технички изазови и како смо их решили

- **Скалирање са једног хоста на више без преписивања.** Већина система намеће чврсту границу између „локалног развоја” и „дистрибуиране продукције”, а прелазак те границе захтева преуређење архитектуре. Ми смо ту границу избрисали системом режима на једном бинарном фајлу: исти слојеви комуницирају путем директних позива у

процесу у режиму `full`, а транспарентно прелазе на `gRPC/SSE/Kafka` у дистрибуираним режимима, тако да је скалирање само питање конфигурације, а не портирања.

- **Непоуздани, ограничени бесплатни провајдери у облаку.** Бесплатни закључак је брз док не дође до грешке 429 или док не нестане усред захтева. Ми смо га учинили поузданим аутоматским откривањем доступних модела, оцењивањем помоћу `LLMsVerifier`, проактивним праћењем заглавља која ограничавају број захтева како бисмо заобишли провајдере који су на ивици гушења, и аутоматским пребацивањем на резерву низ рангирани ланац до локалног `ллама.цпп` – тако да нестабилност тог базена никада не стигне до позиваоца.
- **Компатибилност клијената у два екосистема.** Преписивање клијената како би прихватили нови бацкенд за закључивање није опција. Имплементирали смо и `OpenAI` и `Anthropic` облике `API` – укључујући њихове различите формате стримовања `SSE` – тако да `SDK`-ови из оба табора показују на `HelixLLM` и једноставно раде.

Технолошки стек

- **Go + Gin** — изабрани јер јединствена бинарна датотека са приступом који приоритет даје конкурентност омогућава цео систем модова: једна верзија која може бити серверска апликација на лаптопу или чвор у кластеру. Она носи цео систем и `HTTP` слој гејтвеја.
- **HTTP/3 (QUIC) + TLS 1.3, са HTTP/2 резервним решењем** — изабрани за модеран, нисколатентни и отпоран транспорт веза, изложен као серверска површина са аутоматском негоцијацијом, тако да клијенти који не подржавају `QUIC` тихо прелазе на `HTTP/2`.
- **ллама.цпп (CUDA/Метал/ROCm)** — изабран за преносиву локалну инференцију која убрзава рад на `Nvidia`, `Apple` и `AMD` бацкендовима из једног кода; служи и као гарантовани крајњи резервни провајдер који спречава да ланац резервних решења икада остане без опција.
- **LLMsVerifier** — изабран да претвори питање „који је провајдер тренутно добар“ у број; оцењује и рангира ланац резервних провајдера у облаку на сваких пет минута, тако да усмеравање прати тренутни квалитет, а не застареле претпоставке.
- **Провајдери у облаку (Chutes, OpenRouter, ХуггингФаце, Nvidia, Cerebras, SambaNova, Together)** — изабрани да искористе капацитете бесплатних нивоа услуга код више провајдера; аутоматски се откривају и рангирају у један ланац резервних решења, тако да ниједан провајдер није једина тачка отказа.
- **gRPC + SSE + Kafka** — изабрани као транспортни протоколи између модова за дистрибуиране

имплементације: gRPC за позиве између сервиса, SSE за стримовање, а Kafka за одвојени ток догађаја између улога.

- **Векторско складиште / ембеддингс** — изабрано да покреће RAG пипелине знања од почетка до краја: унос, сегментацију, уградњу и претрагу докумената који осигуравају основу за одговоре модела.
- **Prometheus + OpenTelemetry** — изабрани за метрику и дистрибуирано праћење које прати захтев кроз све распоређене модове.
- **vasic-digital Go потмодули** — изабрани да се поново искористе проверени примитиви инфраструктуре за продукцију уместо да се граде испочетка, чиме се осигурава доследност основе система са ширим стеком.

Статус и напомене о искрености

- **Статус: бета.** Функционалан, активно развијан систем дистрибуиране инференције.
- **Лиценца: није одређена.** У метаподацима репозиторијума није наведена лиценца (licenseInfo нулл) — ово је НЕВЕРИФИКОВАНО и мора бити решено пре него што се наведе лиценца.
- Канонски репозиторијум тренутно води на github.com/HelixDevelopment/llm; путања HelixLLM преусмерава на њега. Подаци о прагу покривености и броју потмодула у РЕАДМЕ фајлу су самопријављени.

Приоритетни ниво: Helix-primary.